

Adaptive Spline Trajectory Planning with Simulated Annealing

1.0 Introduction

The goal of this project was to develop a custom path and motion planner to drive along curved [spline] paths. This project achieves this with a TurtleBot3, navigating a maze and crowded spaces in the ROS2-Gazebo Sim Environment. **Each spline path is trained to adapt and conform to its environment for robot safety when driving.**

2.0 Background

Our robot must navigate from a rest position to a target position. The environment is a PGM map, loaded from a Map Server node. The map is dilated and inflated to create a configuration space (c-space). The c-space is a discretized (vectorized) 2D graph; scaled to fit the continuous space the robot navigates. Each graph cell is either free or occupied by an obstacle or wall. Free spaces are places the robot can safely drive through.

Trajectory planning involves (1) path planning and (2) motion planning. The first is handled with heuristics and an A* graph search on the c-space between where the robot starts and its target position. The result is a connected chain of edges between free cells in the c-space. Common path planners, such as Point-to-Point (P2P), then parse a path from those free cells. However, common planners will follow the A* path closely to ensure robot safety. **This becomes a problem for spline planners.**

2.1 Project Motivation

Our robot creates paths to follow from quintic spline polynomials. Splines create organic and curved geometric paths. These are excellent for non-holonomic motion and smooth maneuvering. Our planner parses waypoints from an A* search to interpolate the spline path.

Splines take on several shapes and forms based on their curvature and the waypoints they interpolate.

The A* search provides potential waypoints. But waypoints must be selectively chosen. As seen in **Figure 1**, a raw path is created from points connected by a dashed line, where a spline interpolates a subset of those points.

This becomes a balancing act. In our case, the A* path is the dashed line. Fewer waypoints means the spline path is smoother and can have larger maneuvers. But, it may be unclear what shape that spline takes on, and if it risks colliding with an obstacle. Conversely, using more waypoints constrains the spline onto the A* path that will ensure it does not collide with obstacles.

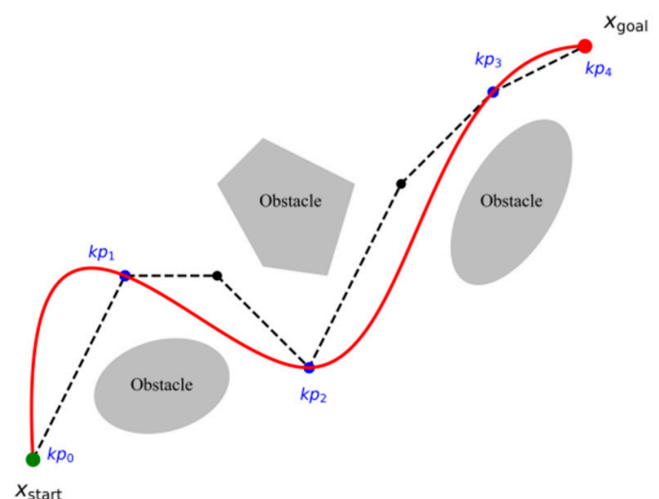


Figure 1: Illustrates spline path planning. [1]

2.2 Exploration-Exploitation Bottleneck on Splines and Safety

If too many waypoints are chosen from the A* path, then A* will have more influence over the path geometry than the curvature of the splines created. The balancing act from before becomes a trajectory bottleneck. The curvature and geometric malleability of splines is minimized when the number of waypoints and safety are maximized. **Figure 1** shows this trade-off, in which every other waypoint is rejected.

The spline path also follows the same homotopy as the original path. Viewing the **Figure 1** from left to right, both the original and spline path travel over the first obstacle and weave between the second and third in the same topological way. However, there are benefits to differing spline homotopy from the original path. For instance, consider if the spline in **Figure 1** could not pass through point **kp2**, and the spline instead traveled over the second obstacle from point **kp1** to **kp3**. **A spline path like that may turn out to be shorter or smoother.**

The robot can guarantee the A* path as completely safe since it has been found. So giving splines the ability to explore different local homotopy or minimize the number of waypoints needed can make safety uncertain. This is especially true in environments that can change. A robot should navigate with A* modeling the overall path, but allow splines the ability to deviate from that path when optimal. **An adaptive spline trajectory planner that avoids obstacles to reconcile exploration with safety solves this problem.**

3.0 Motion Controls

4.0 Adaptive Splines

4.1 Cost-Optimization Problem

4.2

5.0 Trajectory Planning Pipeline

Ignore this bullet list for now. This is temporary notes.

- Adaptive tuning
 - Custom Cost Function
 - Modified Sim Annealing
- Adaptive planning
 - A* -> Parsing -> Centroid Snapping = Waypoints
 - Init waypoint tuning
 - Waypoint Dropout
 - Heading tuning
- Motion Control
 - Trapezoid motion profile on linear velocity
 - PID feedback with a Stanley Controller

6.0 Results

7.0 References

[1] Sun, Z., Luo, Q., Zhang, Z., Peng, Y., Liu, Q., Zheng, S., & Liu, J. (2025). An Integrated Path Planning and Tracking Framework Based on Adaptive Heuristic JPS and B-Spline Optimization. *Machines*, 13(8), 710.

<https://doi.org/10.3390/machines13080710>